

Projet Spring - Gestion Candidats et Formations

Projet Spring — Gestion Candidats & Formations

Étudiant : Ngonde Shadai Bradley

Compte GitHub : [Xenoblade-lab/spring-Ngonde-Shadai-bradley](https://github.com/Xenoblade-lab/spring-Ngonde-Shadai-bradley)

Branche : `main`

Cours : Spring Framework — Module 01 (UPC)

Date : Juillet 2026

Table des matières

1. [Résumé du sujet](#)
 2. [Exigences de l'examen](#)
 3. [État actuel du projet](#)
 4. [Architecture cible](#)
 5. [Modèle de données \(MySQL\)](#)
 6. [Structure des packages et fichiers](#)
 7. [Plan détaillé par phase](#)
 8. [Plan de travail — étapes pas à pas](#)
 9. [Spécifications API REST](#)
 10. [Spécifications interface Web](#)
 11. [Configuration et dépendances](#)
 12. [Checklist de validation finale](#)
 13. [Estimation du temps](#)
-

1. Résumé du sujet

De quoi parle l'application ?

Application de **gestion des inscriptions en formation professionnelle**. Elle permet de :

- Enregistrer des **professions** (métiers visés)
- Gérer des **candidats** (personnes inscrites) liés à une profession
- Proposer des **formations** (programmes de cours)
- Associer candidats et formations via une table de liaison

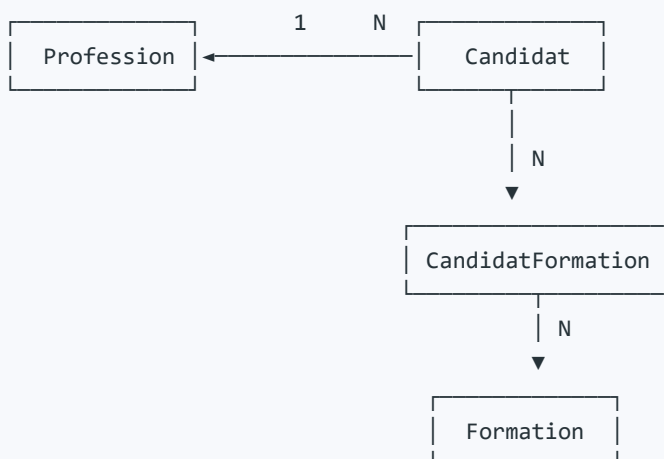
Cas d'usage concret

Un centre de formation enregistre **Bradley Ngonde**, candidat visant la profession **Informaticien**, inscrit aux formations **Spring Framework** (3 mois) et **MySQL avancé** (2 mois).

Entités du domaine

Entité	Description	Exemple
Profession	Métier / filière	Informaticien, Comptable
Candidat	Personne inscrite	Noms, genre, état civil, date/lieu naissance
Formation	Programme de cours	Spring Framework, durée 3 mois
CandidatFormation	Inscription d'un candidat à une formation	Bradley → Spring Framework

Schéma relationnel



2. Exigences de l'examen

Source : docs/spring-projet-dispositions-pratiques-2025-26.pdf

#	Exigence	Comment on la couvre
1	Application Web & API	Thymeleaf (pages HTML) + @RestController (JSON)
2	Architecture 3 couches	@Controller / @Service / @Repository
3	BDD ≥ 3 tables + jointures	4 tables : professions , candidats , formations , candidats_ formations
4	JdbcClient + simpleflatmapper	Repositories avec requêtes JOIN mappées via SFM
5	CRUD complet	Create, Read, Update, Delete sur chaque entité principale

Hors scope (bonus, non exigé pour la note)

- Architecture multi-modules Maven (App-03 du cours)
- Spring Security / JWT
- Déploiement sur serveur externe

3. État actuel du projet

Déjà fait 

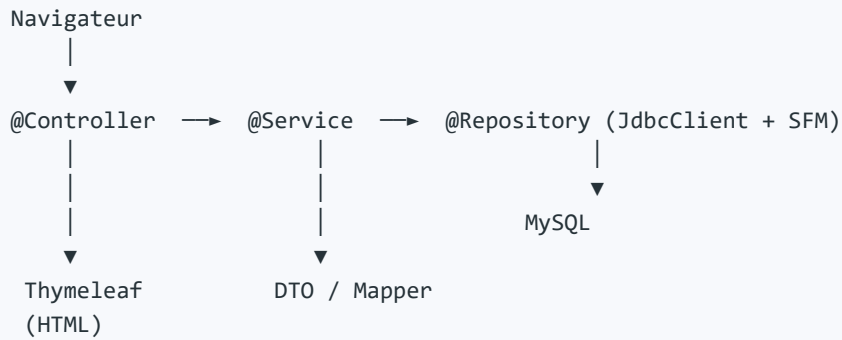
Élément	Fichier(s)	Statut
Projet Spring Boot 4 / Java 21	<code>pom.xml</code>	
Modèles POJO	<code>Candidat</code> , <code>Profession</code> , <code>Formation</code> , <code>CandidatFormation</code>	 (sans JPA)
Formation — Repository	<code>FormationRepoCustom</code> , <code>FormationRepoImpl</code> (<code>JdbcClient</code>)	
Formation — Service	<code>FormationService</code> , <code>FormationServiceImpl</code>	
Formation — API REST	<code>FormationRestController</code>	
Formation — DTO	<code>FormationDto</code>	
Utilitaires	<code>Mapper</code> , <code>MyLibraryUtil</code>	 (partiel)
Config BDD	<code>application.properties</code>	 partiel

À faire 

Élément	Priorité
Dépendance <code>simpleflatmapper</code>	 Haute
Script SQL (schéma + données test)	 Haute
Repositories avec JOIN (SFM)	 Haute
Profession — Service / API / Web	 Haute
Candidat — Service / API / Web	 Haute
CandidatFormation — Service / API / Web	 Moyenne
DTOs manquants	 Haute
Mappers complets	 Haute
Controllers Web (<code>@Controller</code>)	 Haute
Templates Thymeleaf	 Haute
Internationalisation (FR/EN)	 Moyenne
Gestion des exceptions	 Moyenne
Barre de recherche	 Basse
Profils dev/prod	 Basse

4. Architecture cible

Flux d'une requête Web



Flux d'une requête API



Pattern Repository (comme le cours)

Pour chaque entité principale :

XxxRepoCustom.java	→ interface (contrat)
XxxRepoImpl.java	→ @Repository (JdbcClient + simpleflatmapper)

Note : on n'utilise **pas** JpaRepository pour le CRUD principal. JdbcClient + SFM est l'approche exigée par le prof.

5. Modèle de données (MySQL)

Script SQL à créer : `docs/sql/schema.sql`

```
CREATE DATABASE IF NOT EXISTS demo_lmd_2526_01
  CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;

USE demo_lmd_2526_01;

-- Table 1 : Professions
CREATE TABLE professions (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  description VARCHAR(45) NOT NULL,
  UNIQUE KEY uk_professions_description (description)
);

-- Table 2 : Formations
CREATE TABLE formations (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  description VARCHAR(45) NOT NULL,
  duree       INT NOT NULL,
  UNIQUE KEY uk_formations_description (description)
);

-- Table 3 : Candidats
CREATE TABLE candidats (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  profession_pk BIGINT NOT NULL,
  noms        VARCHAR(100) NOT NULL,
  genre       CHAR(1) NOT NULL,           -- M / F
  etat_civil  VARCHAR(20) NOT NULL,
  lieu_nais   VARCHAR(45) NOT NULL,
  date_nais   DATE NOT NULL,
  CONSTRAINT fk_candidats_profession
    FOREIGN KEY (profession_pk) REFERENCES professions(id)
);

-- Table 4 : Liaison candidat ↔ formation
CREATE TABLE candidats_formations (
  id          BIGINT AUTO_INCREMENT PRIMARY KEY,
  candidat_pk BIGINT NOT NULL,
  formation_pk BIGINT NOT NULL,
  CONSTRAINT fk_cf_candidat
    FOREIGN KEY (candidat_pk) REFERENCES candidats(id),
  CONSTRAINT fk_cf_formation
    FOREIGN KEY (formation_pk) REFERENCES formations(id),
  UNIQUE KEY uk_candidat_formation (candidat_pk, formation_pk)
);

-- Index
CREATE INDEX idx_candidats_profession ON candidats(profession_pk);
CREATE INDEX idx_cf_candidat ON candidats_formations(candidat_pk);
CREATE INDEX idx_cf_formation ON candidats_formations(formation_pk);
```

Données de test : `docs/sql/data.sql`

```
INSERT INTO professions (description) VALUES
('Informaticien'), ('Comptable'), ('Infirmier');

INSERT INTO formations (description, duree) VALUES
('Spring Framework', 3), ('MySQL avancé', 2), ('Comptabilité générale', 6);

INSERT INTO candidats (profession_pk, noms, genre, etat_civil, lieu_nais, date_nais) VALUES
(1, 'Ngonde Shadai Bradley', 'M', 'Célibataire', 'Kinshasa', '2000-05-15'),
(2, 'Mbuyi Grace', 'F', 'Mariée', 'Lubumbashi', '1998-11-20');

INSERT INTO candidats_formations (candidat_pk, formation_pk) VALUES
(1, 1), (1, 2), (2, 3);
```

Requêtes avec jointures (exemples clés)

Candidat + Profession :

```
SELECT c.*, p.id AS profession_id, p.description AS profession_description
FROM candidats c
INNER JOIN professions p ON c.profession_pk = p.id
ORDER BY c.noms;
```

Candidat + Formations :

```
SELECT c.*, f.id AS formation_id, f.description AS formation_description, f.duree
FROM candidats c
INNER JOIN candidats_formations cf ON c.id = cf.candidat_pk
INNER JOIN formations f ON cf.formation_pk = f.id
WHERE c.id = ?
```

Ces requêtes seront implémentées dans les `@Repository` avec `simpleflatmapper` pour mapper automatiquement les objets imbriqués.

6. Structure des packages et fichiers

```
src/main/java/edu/upc/
├─ DemoUpc252602Application.java
├─ config/
│   └─ AppConfig.java ← i18n, LocaleResolver, intercepteur langue
├─ controllers/
│   ├── ProfessionController.java ← Web Thymeleaf
│   ├── CandidatController.java
│   ├── FormationController.java
│   ├── CandidatFormationController.java
│   ├── HomeController.java ← page d'accueil
│   └─ api/
│       ├── ProfessionRestController.java
│       ├── CandidatRestController.java
│       ├── FormationRestController.java ← ✓ existe
│       └─ CandidatFormationRestController.java
├─ services/
│   ├── ProfessionService.java + Impl
│   ├── CandidatService.java + Impl
│   ├── FormationService.java + Impl ← ✓ existe
│   └─ CandidatFormationService.java + Impl
├─ repositories/
│   ├── ProfessionRepoCustom.java + Impl
│   ├── CandidatRepoCustom.java + Impl ← requêtes JOIN ici
│   ├── FormationRepoCustom.java + Impl ← ✓ existe (à migrer SFM)
│   └─ CandidatFormationRepoCustom.java + Impl
├─ models/
│   ├── Profession.java
│   ├── Candidat.java ← ajouter champ Profession profession
│   ├── Formation.java
│   ├── CandidatFormation.java
│   └─ dtos/
│       ├── ProfessionDto.java
│       ├── CandidatDto.java
│       ├── FormationDto.java ← ✓ existe
│       └─ CandidatFormationDto.java
├─ utils/
│   ├── Mapper.java ← étendre pour toutes les entités
│   ├── MyLibraryUtil.java
│   ├── MyDateUtils.java ← conversion String ↔ Date
│   └─ exceptions/
│       ├── NotFoundException.java
│       └─ GlobalExceptionHandler.java ← @ControllerAdvice + @RestControllerAdvice

src/main/resources/
├─ application.properties
├─ application-dev.properties
├─ Bundle.properties ← messages EN
├─ Bundle_fr.properties ← messages FR
├─ static/
│   └─ css/
│       ├── bootstrap.min.css
│       └─ app.css
├─ templates/
│   ├── layout.html ← layout Thymeleaf (optionnel)
│   └─ index.html ← accueil
```

```
├─ professions-view.html
├─ professions-form.html
├─ candidats-view.html
├─ candidats-form.html
├─ formations-view.html
├─ formations-form.html
└─ candidats-formations-view.html
```

7. Plan détaillé par phase

Phase 0 — Préparation (Jour 1, ~1h)

Objectif : Environnement prêt, BDD créée, dépendances ajoutées.

- ☐ Vérifier Java 21, Maven, MySQL (Laragon port 3307)
- ☐ Exécuter `docs/sql/schema.sql` et `docs/sql/data.sql`
- ☐ Ajouter `simpleflatmapper` et `spring-boot-starter-jdbc` au `pom.xml`
- ☐ Compléter `application.properties` (port, dialect, profils)
- ☐ Vérifier que l'app démarre : `./mvnw spring-boot:run`

Phase 1 — Couche Data / Repositories (Jour 1-2, ~4h)

Objectif : Tous les `@Repository` fonctionnels avec `JdbcTemplate` + `simpleflatmapper`.

1.1 Profession

- Interface `ProfessionRepoCustom` : `create`, `update`, `delete`, `getById`, `get`
- Impl `ProfessionRepoImpl` avec `JdbcTemplate`

1.2 Formation (refactoring)

- Migrer `FormationRepoImpl` pour utiliser `simpleflatmapper` au lieu du mapping natif `JdbcTemplate`
- Garder les mêmes méthodes CRUD

1.3 Candidat (avec jointures)

- Interface `CandidatRepoCustom` : CRUD + `getWithProfession()`, `getWithFormations(id)`, `search(keyword)`
- Impl avec requête JOIN candidats ↔ professions via SFM
- Modèle enrichi : `Candidat` contient un objet `Profession` (pour le mapping SFM)

1.4 CandidatFormation

- CRUD pour associer / dissocier candidat ↔ formation
- Requête JOIN pour lister les inscriptions avec noms candidat + formation

Phase 2 — Couche Service + DTOs (Jour 2, ~3h)

Objectif : Logique métier et validation.

- ☐ Créer tous les DTOs avec annotations `@NotBlank`, `@Size`, `@Positive`, etc.
 - ☐ Étendre `Mapper` pour toutes les entités
 - ☐ Créer `ProfessionService` + `ProfessionServiceImpl`
 - ☐ Créer `CandidatService` + `CandidatServiceImpl`
 - ☐ Créer `CandidatFormationService` + `CandidatFormationServiceImpl`
 - ☐ Ajouter `@Transactional` sur create/update/delete
 - ☐ Créer exceptions `NotFoundException` + handler global
-

Phase 3 — API REST (Jour 2–3, ~2h)

Objectif : Tous les endpoints JSON testables avec Postman.

Méthode	Endpoint	Action
GET	<code>/api/professions</code>	Liste
GET	<code>/api/professions/{id}</code>	Détail
POST	<code>/api/professions</code>	Créer
PUT	<code>/api/professions/{id}</code>	Modifier
DELETE	<code>/api/professions/{id}</code>	Supprimer
GET	<code>/api/candidats</code>	Liste (avec profession)
GET	<code>/api/candidats/{id}</code>	Détail + formations
POST	<code>/api/candidats</code>	Créer
PUT	<code>/api/candidats/{id}</code>	Modifier
DELETE	<code>/api/candidats/{id}</code>	Supprimer
GET	<code>/api/formations</code>	Liste
...	...	(déjà partiellement fait)
GET	<code>/api/candidats-formations</code>	Liste inscriptions
POST	<code>/api/candidats-formations</code>	Inscrire candidat
DELETE	<code>/api/candidats-formations/{id}</code>	Désinscrire

Phase 4 — Interface Web Thymeleaf (Jour 3–4, ~5h)

Objectif : Application navigable dans le navigateur.

4.1 Configuration

- ☐ `AppConfig` : `MessageSource`, `LocaleResolver`, intercepteur `?locale=fr|en`
- ☐ Fichiers `Bundle.properties` / `Bundle_fr.properties`
- ☐ Bootstrap CSS dans `static/css/`

4.2 Pages par entité (pattern identique pour chaque)

- **Vue liste** (`xxx-view.html`) : tableau + boutons Ajouter / Éditer / Supprimer + recherche
- **Formulaire** (`xxx-form.html`) : champs validés + messages d'erreur

4.3 Controllers Web

- `@Controller` avec mappings :
 - `GET /professions` → liste
 - `GET /professions/create` → formulaire vide
 - `GET /professions/{id}/edit` → formulaire pré-rempli
 - `POST /professions/save` → enregistrer
 - `GET /professions/{id}/delete` → supprimer
- Répéter pour Candidat, Formation, CandidatFormation

4.4 Page d'accueil

- `GET /` → dashboard avec liens vers chaque module

Phase 5 — Finitions (Jour 4–5, ~2h)

Objectif : Qualité et conformité examen.

- ☐ Barre de recherche sur candidats (paramètre `keyword`)
- ☐ Messages flash de confirmation (succès / erreur)
- ☐ Gestion des erreurs (404, validation)
- ☐ Tests manuels Postman (collection exportée)
- ☐ Tests manuels navigateur (tous les CRUD)
- ☐ Commit + push sur GitHub
- ☐ Relire checklist finale (section 12)

8. Plan de travail — étapes pas à pas

Cocher chaque étape au fur et à mesure. Ordre strict recommandé.

Étape 1 — Base de données

- ☐ **1.1** Créer le fichier `docs/sql/schema.sql`
- ☐ **1.2** Créer le fichier `docs/sql/data.sql`
- ☐ **1.3** Exécuter les scripts dans MySQL (HeidiSQL / Laragon)
- ☐ **1.4** Vérifier les 4 tables et les données de test

Étape 2 — Dépendances Maven

- ☐ 2.1 Ajouter `simpleflatmapper-jdbc` dans `pom.xml`
- ☐ 2.2 Ajouter `spring-boot-starter-jdbc` si absent
- ☐ 2.3 Lancer `./mvnw clean compile` sans erreur

Étape 3 — Configuration Spring

- ☐ 3.1 Compléter `application.properties` (URL BDD, port 8081, logging)
- ☐ 3.2 Créer `application-dev.properties`
- ☐ 3.3 Configurer `spring.profiles.active=dev`

Étape 4 — Modèles et DTOs

- ☐ 4.1 Enrichir `Candidat` avec champ `Profession profession` (pour SFM)
- ☐ 4.2 Créer `ProfessionDto`
- ☐ 4.3 Créer `CandidatDto`
- ☐ 4.4 Créer `CandidatFormationDto`
- ☐ 4.5 Étendre `Mapper` pour toutes les conversions

Étape 5 — Repository Profession

- ☐ 5.1 Créer `ProfessionRepoCustom.java`
- ☐ 5.2 Créer `ProfessionRepoImpl.java` (`JdbcClient`)
- ☐ 5.3 Tester manuellement via un `@SpringBootTest` ou log temporaire

Étape 6 — Repository Formation (migration SFM)

- ☐ 6.1 Configurer `JdbcMapper` / `ResultSetExtractor` avec `simpleflatmapper`
- ☐ 6.2 Refactorer `FormationRepoImpl` pour utiliser SFM
- ☐ 6.3 Vérifier que l'API Formation existante fonctionne toujours

Étape 7 — Repository Candidat (jointures)

- ☐ 7.1 Créer `CandidatRepoCustom.java` (CRUD + `getWithProfession` + `search`)
- ☐ 7.2 Créer `CandidatRepoImpl.java` avec requête JOIN + SFM
- ☐ 7.3 Tester la récupération candidat + profession

Étape 8 — Repository CandidatFormation

- ☐ 8.1 Créer `CandidatFormationRepoCustom.java`
- ☐ 8.2 Créer `CandidatFormationRepoImpl.java` avec JOIN
- ☐ 8.3 Tester inscription / désinscription

Étape 9 — Services

- ☐ 9.1 `ProfessionService` + `ProfessionServiceImpl`
- ☐ 9.2 `CandidatService` + `CandidatServiceImpl`
- ☐ 9.3 Vérifier `FormationService` (déjà existant)

- ☐ 9.4 `CandidatFormationService` + `CandidatFormationServiceImpl`
- ☐ 9.5 Créer `NotFoundException` + `GlobalExceptionHandler`

Étape 10 — API REST

- ☐ 10.1 `ProfessionRestController` (CRUD complet)
- ☐ 10.2 `CandidatRestController` (CRUD + liste avec profession)
- ☐ 10.3 Vérifier `FormationRestController` (existant)
- ☐ 10.4 `CandidatFormationRestController`
- ☐ 10.5 Tester tous les endpoints avec Postman

Étape 11 — Internationalisation

- ☐ 11.1 Créer `AppConfig.java` (`MessageSource`, `LocaleResolver`)
- ☐ 11.2 Créer `Bundle.properties` et `Bundle_fr.properties`
- ☐ 11.3 Tester `http://localhost:8081/?locale=fr`

Étape 12 — Interface Web — Profession

- ☐ 12.1 `ProfessionController.java`
- ☐ 12.2 `professions-view.html`
- ☐ 12.3 `professions-form.html`
- ☐ 12.4 Tester CRUD complet dans le navigateur

Étape 13 — Interface Web — Formation

- ☐ 13.1 `FormationController.java`
- ☐ 13.2 `formations-view.html`
- ☐ 13.3 `formations-form.html`
- ☐ 13.4 Tester CRUD complet

Étape 14 — Interface Web — Candidat

- ☐ 14.1 `CandidatController.java`
- ☐ 14.2 `candidats-view.html` (afficher profession via jointure)
- ☐ 14.3 `candidats-form.html` (select profession)
- ☐ 14.4 Barre de recherche par nom
- ☐ 14.5 Tester CRUD complet

Étape 15 — Interface Web — CandidatFormation

- ☐ 15.1 `CandidatFormationController.java`
- ☐ 15.2 Page liste des inscriptions
- ☐ 15.3 Formulaire d'inscription (select candidat + formation)
- ☐ 15.4 Tester ajout / suppression

Étape 16 — Accueil et finitions

- ☐ 16.1 HomeController + index.html (menu navigation)
 - ☐ 16.2 CSS Bootstrap + app.css
 - ☐ 16.3 Messages flash (succès / erreur)
 - ☐ 16.4 Revue complète checklist section 12
 - ☐ 16.5 Commit final + push GitHub
-

9. Spécifications API REST

Format de réponse

Succès :

```
HTTP 200 OK
{ "id": 1, "description": "Spring Framework", "duree": 3 }
```

Création :

```
HTTP 201 CREATED
{ "id": 4, "description": "Angular", "duree": 2 }
```

Erreur validation :

```
HTTP 400 BAD REQUEST
{ "message": "Description invalide, Duree invalide" }
```

Exemple Postman — Créer une formation

```
POST http://localhost:8081/api/formations
Content-Type: application/json

{
  "description": "Angular",
  "duree": 2
}
```

Exemple Postman — Lister candidats avec profession

```
GET http://localhost:8081/api/candidats
```

Réponse attendue :

```
[
  {
    "id": 1,
    "noms": "Ngonde Shadai Bradley",
    "genre": "M",
    "profession": {
      "id": 1,
      "description": "Informaticien"
    }
  }
]
```

10. Spécifications interface Web

Navigation

```
Accueil (/)
├─ Professions (/professions)
├─ Candidats (/candidats)
├─ Formations (/formations)
└─ Inscriptions (/candidats-formations)
```

Pattern page liste

- Tableau Bootstrap avec colonnes pertinentes
- Bouton **Nouveau** → formulaire création
- Colonne **Actions** : Éditer | Supprimer
- Champ **Recherche** (sur candidats minimum)
- Lien **FR** | **EN** dans le header

Pattern formulaire

- Champs avec validation côté serveur
- Affichage des erreurs sous chaque champ (`th:errors`)
- Boutons **Enregistrer** et **Annuler**
- Select pour les clés étrangères (profession, candidat, formation)

11. Configuration et dépendances

Dépendance simpleflatmapper à ajouter

```
<dependency>
  <groupId>org.simpleflatmapper</groupId>
  <artifactId>sfm-springjdbc</artifactId>
  <version>8.2.3</version>
</dependency>
```

application.properties cible

```
spring.application.name=spring-Ngonde-Shadai-bradley
spring.profiles.active=dev
server.port=8081
```

application-dev.properties cible

```
spring.datasource.url=jdbc:mysql://localhost:3307/demo_lmd_2526_01?
useSSL=false&allowPublicKeyRetrieval=true
spring.datasource.username=root
spring.datasource.password=
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
logging.level.edu.upc=DEBUG
```

12. Checklist de validation finale

Avant de rendre le projet, vérifier :

Exigences examen

- ☐ L'application démarre sans erreur (`./mvnw spring-boot:run`)
- ☐ Interface Web accessible sur `http://localhost:8081`
- ☐ API REST testable sur `http://localhost:8081/api/...`
- ☐ Architecture 3 couches visible dans le code
- ☐ Au moins 3 tables MySQL avec données
- ☐ Au moins une requête SQL avec JOIN fonctionnelle
- ☐ JdbcClient utilisé dans les `@Repository`
- ☐ simpleflatmapper utilisé pour le mapping JOIN
- ☐ CRUD complet sur Profession, Candidat, Formation

Qualité

- ☐ DTOs séparés des modèles (pas d'exposition directe des entités)
- ☐ Validation des formulaires (annotations + messages)
- ☐ Gestion des erreurs (404, validation)
- ☐ Code poussé sur GitHub (branche `main`)
- ☐ Pas de credentials en dur dans le code source public

Démonstration orale (si présentation)

- ☐ Savoir expliquer le flux MVC
- ☐ Savoir montrer une requête JOIN dans le code
- ☐ Savoir tester un endpoint Postman en direct
- ☐ Savoir expliquer le rôle de simpleflatmapper

13. Estimation du temps

Phase	Durée estimée	Cumul
Phase 0 — Préparation	1h	1h
Phase 1 — Repositories	4h	5h
Phase 2 — Services + DTOs	3h	8h
Phase 3 — API REST	2h	10h
Phase 4 — Web Thymeleaf	5h	15h
Phase 5 — Finitions	2h	17h

Total estimé : ~17 heures (2–3 jours de travail concentré, ou 1 semaine à raison de 2–3h/jour)

Références

- Consignes examen : [docs/spring-projet-dispositions-pratiques-2025-26.pdf](#)
 - Cours Spring : [docs/spring-lmd3-01.pdf](#)
 - simpleflatmapper : <https://simpleflatmapper.org>
 - Spring Boot 4 docs : <https://docs.spring.io/spring-boot/docs/current/reference/html/>
 - Thymeleaf : <https://www.thymeleaf.org>
-

Document généré pour le projet individuel Spring — UPC LMD3